

UNIT-II

SOFTWARE REQUIREMENTS

IEEE defines Requirement as :

1. A condition or capability needed by a user to solve a problem or achieve an objective
2. A condition or capability that must be met or possessed by a system or a system component to satisfy contract, standard, specification or formally imposed document
3. A documented representation of a condition nor capability as in 1 or 2

SOFTWARE REQUIREMENTS

- Encompasses both the User's view of the requirements(the external view) and the Developer's view(inside characteristics)

User's Requirements

--Statements in a natural language plus diagram, describing the services the system is expected to provide and the constraints

- System Requirements --Describe the system's function, services and operational condition

SOFTWARE REQUIREMENTS

- System Functional Requirements
 - Statement of services the system should provide
 - Describe the behavior in particular situations
 - Defines the system reaction to particular inputs
- Nonfunctional Requirements
 - Constraints on the services or functions offered by the system
 - Include timing constraints, constraints on the development process and standards
 - Apply to system as a whole
- Domain Requirements
 - Requirements relate to specific application of the system
 - Reflect characteristics and constraints of that system

FUNCTIONAL REQUIREMENTS

- Should be both complete and consistent
- Completeness
 - All services required by the user should be defined
- Consistent
 - Requirements should not have contradictory definition
- Difficult to achieve completeness and consistency for large system

NON-FUNCTIONAL REQUIREMENTS

Types of Non-functional Requirements

1.Product Requirements

-Specify product behavior
-Include the following

- Usability
- Efficiency
- Reliability
- Portability

2. Organizational Requirements

--Derived from policies and procedures

--Include the following:

- Delivery
- Implementation
- Standard

3. External Requirements

-- Derived from factors external to the system and its development process

--Includes the following

- Interoperability
- Ethical
- Legislative
-

PROBLEMS FACED USING THE NATURAL LANGUAGE

1. Lack of clarity-- Leads to misunderstanding because of ambiguity of natural language
2. Confusion-- Due to over flexibility, sometime difficult to find whether requirements are same or distinct.
3. Amalgamation problem-- Difficult to modularize natural language requirements

STRUCTURED LANGUAGE SPECIFICATION

- Requirements are written in a standard way
- Ensures degree of uniformity
- Provide templates to specify system requirements
- Include control constructs and graphical highlighting to partition the specification

SYSTEM REQUIREMENTS STANDARD FORM

- Function
- Description
- Inputs
- Source
- Outputs
- Destination
- Action
- Precondition
- Post condition
- Side effects

Interface Specification

- Working of new system must match with the existing system
- Interface provides this capability and precisely specified

Three types of interfaces

1. Procedural interface-- Used for calling the existing programs by the new programs
2. Data structures-- Provide data passing from one sub-system to another
3. Representations of Data-- Ordering of bits to match with the existing system

--Most common in real-time and embedded system

The Software Requirements document

The requirements document is the official statement of what is required of the system developers. Should include both a definition of user requirements and a specification of the system requirements. It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it

The Software Requirements document

Suggests that there are 6 requirements that requirement document should satisfy. It should

- specify only external system behavior
- Specify constraints on the implementation.
- Be easy to change
- Serve as reference tool for system maintainers
- Record forethought about the life cycle of the system.
- Characterize acceptable responses to undesired events

Purpose of SRS

- Communication between the Customer, Analyst, system developers, maintainers,
- firm foundation for the design phase
- support system testing activities
- Support project management and control
- controlling the evolution of the system

IEEE requirements standard

Defines a generic structure for a requirements document that must be instantiated for each specific system.

- Introduction.
- General description.
- Specific requirements.
- Appendices.
- Index.

IEEE requirements standard

1. Introduction Purpose

Scope

Definitions, Acronyms and Abbreviations References

Overview

4. General description Product perspective Product function summary User characteristics General constraints

Assumptions and dependencies

5. Specific Requirements

- Functional requirements

6. External interface requirements

- Performance requirements

- Design constraints

- Attributes eg. security, availability, maintainability, transferability/conversion

- Other requirements

• Appendices

• Index

REQUIREMENTS ENGINEERING PROCESS

To create and maintain a system requirement document. The overall process includes four high level requirements engineering sub-processes:

1. Feasibility study

--Concerned with assessing whether the system is useful to the business

2. Elicitation and analysis

--Discovering requirements

3. Specifications

--Converting the requirements into a standard form

4. Validation

-- Checking that the requirements actually define the system that the customer wants

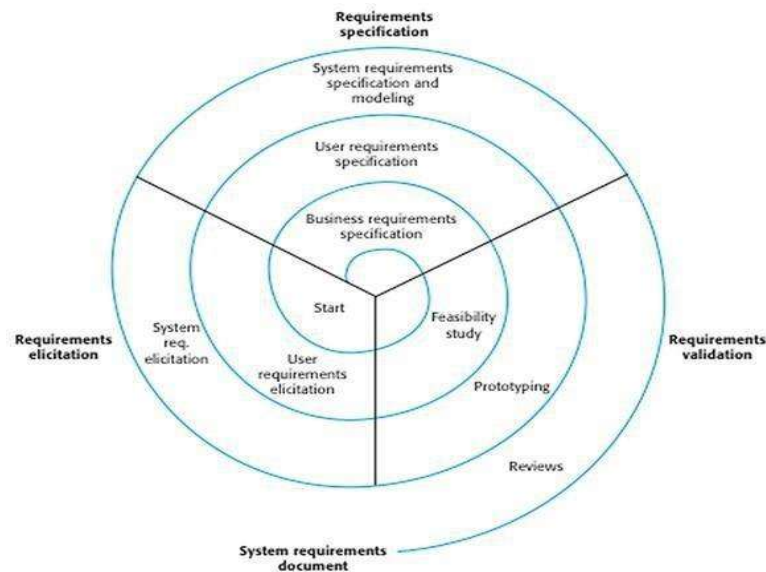
SPIRAL REPRESENTATION OF REQUIREMENTS ENGINEERING PROCESS

Process represented as three stage activity. Activities are organized as an iterative process around a spiral. Early in the process, most effort will be spent on understanding high-level business and the use requirement. Later in the outer rings, more effort will be devoted to system requirements engineering and system modeling

Three level process consists of: 1. Requirements elicitation

2. Requirements specification

3. Requirements validation



FEASIBILITY STUDIES

Starting point of the requirements engineering process

- Input: Set of preliminary business requirements, an outline description of the system and how the system is intended to support business processes
- Output: Feasibility report that recommends whether or not it is worth carrying out further

Feasibility report answers a number of questions:

1. Does the system contribute to the overall objective
2. Can the system be implemented using the current technology and within given cost and schedule
3. Can the system be integrated with other system which are already in place.

REQUIREMENTS ELICITATION ANALYSIS

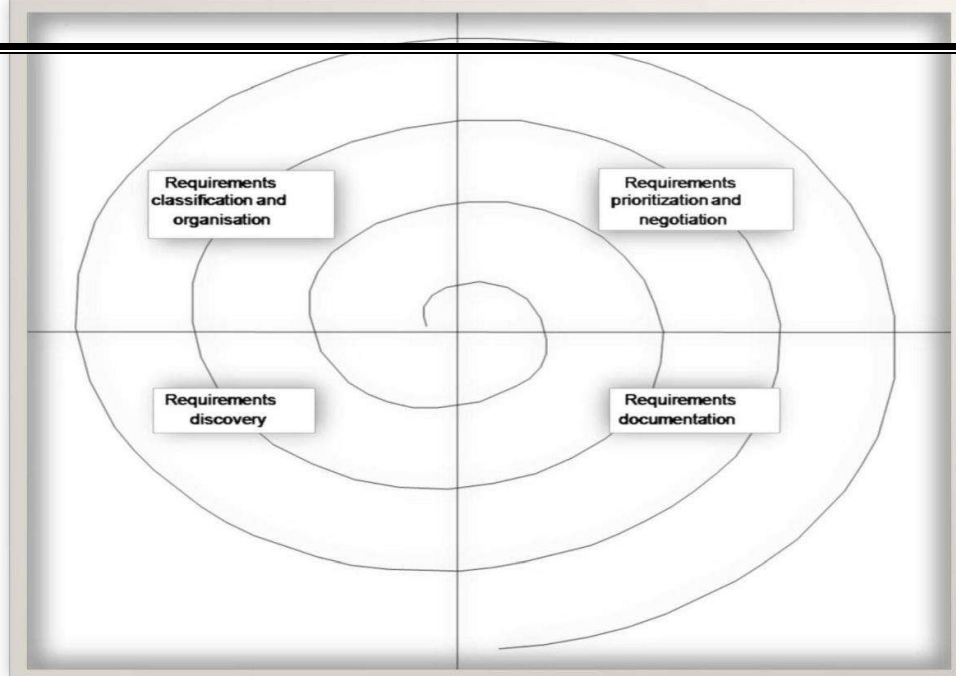
Involves a number of people in an organization.

Stakeholder definition-- Refers to any person or group who will be affected by the system directly or indirectly i.e. End-users, Engineers, business managers, domain experts.

Reasons why eliciting is difficult

1. Stakeholder often don't know what they want from the computer system.
2. Stakeholder expression of requirements in natural language is sometimes difficult to Understand.
3. Different stakeholders express requirements differently
4. Influences of political factors Change in requirements due to dynamic environments.

REQUIREMENTS ELICITATION PROCESS



Process activities

1. Requirement Discovery -- Interaction with stakeholder to collect their requirements including domain and documentation
2. Requirements classification and organization -- Coherent clustering of requirements from unstructured collection of requirements
3. Requirements prioritization and negotiation -- Assigning priority to requirements
--Resolves conflicting requirements through negotiation
4. Requirements documentation -- Requirements be documented and placed in the next round of spiral
5. The spiral representation of Requirements Engineering

REQUIREMENTS DISCOVERY TECHNIQUES

1. View points --Based on the viewpoints expressed by the stake holder
--Recognizes multiple perspectives and provides a framework for discovering conflicts in the requirements proposed by different stakeholders

Three Generic types of viewpoints

1. Interactor viewpoint--Represents people or other system that interact directly with the system
2. Indirect viewpoint--Stakeholders who influence the requirements, but don't use the system
3. Domain viewpoint--Requirements domain characteristics and constraints that influence the requirements.

2. Interviewing--Puts questions to stakeholders about the system that they use and the system to be developed. Requirements are derived from the answers.

Two types of interview

- Closed interviews where the stakeholders answer a pre-defined set of questions.
- Open interviews discuss a range of issues with the stakeholders for better understanding their needs.

Effective interviewers

- a) Open-minded: no pre-conceived ideas
- b) Prompter: prompt the interviewee to start discussion with a question or a proposal

3. Scenarios --Easier to relate to real life examples than to abstract description. Starts with an outline

of the interaction and during elicitation, details are added to create a complete description of that interaction

Scenario includes:

1. Description at the start of the scenario
2. Description of normal flow of the event
3. Description of what can go wrong and how this is handled
4. Information about other activities parallel to the scenario
5. Description of the system state when the scenario finishes

LIBSYS scenario

- **Initial assumption:** The user has logged on to the LIBSYS system and has located the journal containing the copy of the article.
- **Normal:** The user selects the article to be copied. He or she is then prompted by the system to either provide subscriber information for the journal or to indicate how they will pay for the article. Alternative payment methods are by credit card or by quoting an organizational account number.
- The user is then asked to fill in a copyright form that maintains details of the transaction and they then submit this to the LIBSYS system.
- The copyright form is checked and, if OK, the PDF version of the article is downloaded to the LIBSYS working area on the user's computer and the user is informed that it is available. The user is asked to select a printer and a copy of the article is printed

LIBSYS scenario

What can go wrong: The user may fail to fill in the copyright form correctly. In this case, the form should be re-presented to the user for correction. If the resubmitted form is still incorrect then the user's request for the article is rejected.

- The payment may be rejected by the system. The user's request for the article is rejected.
- The article download may fail. Retry until successful or the user terminates the session..

Other activities: Simultaneous downloads of other articles.

System state on completion: User is logged on. The downloaded article has been deleted from LIBSYS workspace if it has been flagged as print-only.

4. Use cases -- scenario based technique for requirement elicitation. A fundamental feature of UML, notation for describing object-oriented system models. Identifies a type of interaction and the actors involved. Sequence diagrams are used to add information to a Use case

Article printing use-case Article printing LIBSYS use cases Article printing Article search

User administration Supplier Catalogue services Library

User Library Staff

REQUIREMENTS VALIDATION

Concerned with showing that the requirements define the system that the customer wants. Important because errors in requirements can lead to extensive rework cost

Validation checks

1. Validity checks --Verification that the system performs the intended function by the user
2. Consistency check --Requirements should not conflict
3. Completeness checks --Includes requirements which define all functions and constraints intended by the system user
4. Realism checks --Ensures that the requirements can be actually implemented
5. Verifiability -- Testable to avoid disputes between customer and developer.

VALIDATION TECHNIQUES

1. REQUIREMENTS REVIEWS

Reviewers check the following:

- (a) Verifiability: Testable
- (b) Comprehensibility
- (c) Traceability
- (d) Adaptability

2. PROTOTYPING

3. TEST-CASE GENERATION

Requirements management
Requirements are likely to change for large software systems and as such requirements management process is required to handle changes.

Reasons for requirements changes

- (a) Diverse Users community where users have different requirements and priorities
 - (b) System customers and end users are different
 - (c) Change in the business and technical environment after installation
- Two classes of requirements
- (a) **Enduring requirements:** Relatively stable requirements
 - (b) **Volatile requirements:** Likely to change during system development process or during operation

Requirements management planning

An essential first stage in requirement management process. Planning process consists of the following

- 1. Requirements identification -- Each requirement must have unique tag for cross reference and traceability
- 2. Change management process -- Set of activities that assess the impact and cost of changes
- 3. Traceability policy -- A matrix showing links between requirements and other elements of software development
- 4. CASE tool support -- Automatic tool to improve efficiency of change management process. Automated tools are required for requirements storage, change management and traceability management

Traceability

Maintains three types of traceability information.

- 1. Source traceability--Links the requirements to the stakeholders
- 2. Requirements traceability--Links dependent requirements within the requirements document
- 3. Design traceability-- Links from the requirements to the design module

Req. id	1.1	1.2	1.3	2.1	2.2	2.3	3.1	3.2
1.1		D	R					
1.2			D			D		D
1.3	R			R				
2.1			R		D			D
2.2								D
2.3		R		D				
3.1								R
3.2							R	

A traceability matrix Requirements change management consists of three principal stages:

- 1. Problem analysis and change specification-- Process starts with a specific change proposal and analysed to verify that it is valid

2. Change analysis and costing--Impact analysis in terms of cost, time and risks
3. Change implementation--Carrying out the changes in requirements document, system design and its implementation

SYSTEM MODELS

Used in analysis process to develop understanding of the existing system or new system. Excludes details. An abstraction of the system

Types of system models

1. Context models
2. Behavioural models
3. Data models
4. Object models
5. Structured models

CONTEXT MODELS

A type of architectural model. Consists of sub-systems that make up an entire system

First step: To identify the subsystem.

Represent the high level architectural model as simple block diagram

- Depict each sub system a named rectangle
 - Lines between rectangles indicate associations between subsystems
- Disadvantages
--Concerned with system environment only, doesn't take into account other systems, which may take data or give data to the model

The context of an ATM system consists of the following Auto-teller system Security system Maintenance system Account data base Usage database Branch accounting system Branch counter system

Behavioral models

Describes the overall behaviour of a system. Two types of behavioural model

1. Data Flow models
2. State machine models

Data flow models --Concentrate on the flow of data and functional transformation on that data. Show the processing of data and its flow through a sequence of processing steps. Help analyst understand what is going on

Advantages

- Simple and easily understandable
- Useful during analysis of requirements

State machine models

Describe how a system responds to internal or external events. Shows system states and events that cause transition from one state to another. Does not show the flow of data within the system. Used for modeling of real time systems

Exp: Microwave oven

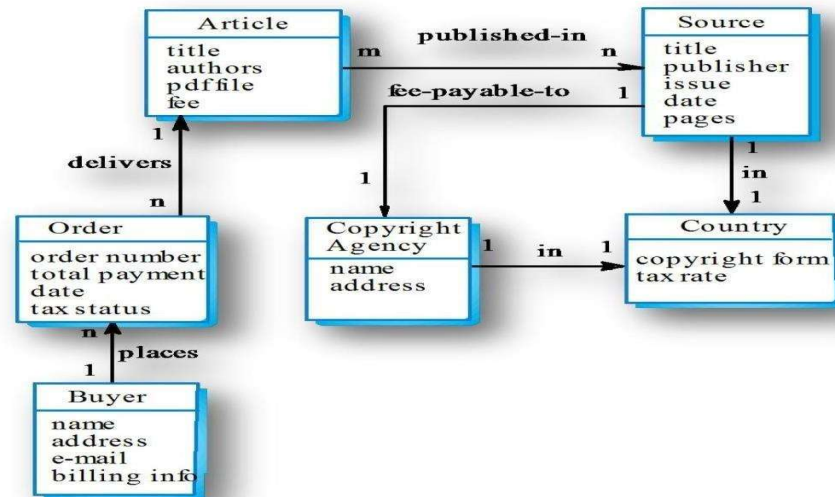
Assumes that at any time, the system is in one of a number of possible states. Stimulus triggers a transition from one state to another state

Disadvantage

- Number of possible states increases rapidly for large system models

DATA MODELS

Used to describe the logical structure of data processed by the system. An entity-relation- attribute model sets out the entities in the system, the relationships between these entities and the entity attributes. Widely used in database design. Can readily be implemented using relational databases. No specific notation provided in the UML but objects and associations can be used.



OBJECT MODELS

An object oriented approach is commonly used for interactive systems development. Expresses the systems requirements using objects and developing the system in an object oriented PL such as c++ A object class: An abstraction over a set of objects that identifies common attributes. Objects are instances of object class. Many objects may be created from a single class.

Analysis process

-- Identifies objects and object classes Object class in UML

--Represented as a vertically oriented rectangle with three sections

- The name of the object class in the top section
- The class attributes in the middle section
- The operations associated with the object class are in lower section.

OBJECT MODELS INHERITANCE MODELS

A type of object oriented model which involves in object classes attributes. Arranges classes into an inheritance hierarchy with the most general object class at the top of hierarchy Specialized objects inherit their attributes and services

UML notation

-- Inheritance is shown upward rather than downward

--Single Inheritance: Every object class inherits its attributes and operations from a single parent class

--Multiple Inheritance: A class of several of several parents.

Name	Description	Type	Date
Article	Details of the published article that may be ordered by people using LIBSYS.	Entity	30.12.2002
authors	The names of the authors of the article who may be due a share of the fee.	Attribute	30.12.2002
Buyer	The person or organisation that orders a copy of the article.	Entity	30.12.2002
fee-payable-to	A 1:1 relationship between Article and the Copyright Agency who should be paid the copyright fee.	Relation	29.12.2002
Address (Buyer)	The address of the buyer. This is used to any paper billing information that is required.	Attribute	31.12.2002

UML Diagrams

The UML is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML gives you a standard way to write a system's blueprints, covering conceptual things, such as business processes and system functions, as well as concrete things, such as classes written in a specific programming language, database schemas, and reusable software components.

Model

A model is a simplification of reality. A model provides the blueprints of a system. A model may be structural, emphasizing the organization of the system, or it may be behavioral, emphasizing the dynamics of the system.

Why do we model

We build models so that we can better understand the system we are developing.

Through modeling, we achieve four aims.

1. Models help us to visualize a system as it is or as we want it to be.
2. Models permit us to specify the structure or behavior of a system.
3. Models give us a template that guides us in constructing a system.
4. Models document the decisions we have made.

We build models of complex systems because we cannot comprehend such a system in its entirety.

Principles of Modeling

There are four basic principles of model

1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.
2. Every model may be expressed at different levels of precision.
3. The best models are connected to reality.
4. No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models.

An Overview of UML

- The Unified Modeling Language is a standard language for writing software blueprints. The UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system.
- The UML is appropriate for modeling systems ranging from enterprise information systems to distributed Web-based applications and even to hard real time embedded systems. It is a very expressive language, addressing all the views needed to develop and then deploy such systems.

The UML is a language for

- Visualizing
- Specifying
- Constructing
- Documenting
- **Visualizing** The UML is more than just a bunch of graphical symbols. Rather, behind each symbol in the UML notation is a well-defined semantics. In this manner, one developer can write a model in the UML, and another developer, or even another tool, can interpret that model unambiguously
- **Specifying** means building models that are precise, unambiguous, and complete.
- **Constructing** the UML is not a visual programming language, but its models can be directly connected to a variety of programming languages
- **Documenting** a healthy software organization produces all sorts of artifacts in addition to raw executable code. These artifacts include
 - Requirements
 - Architecture
 - Design
 - Source code
 - Project plans
 - Tests
 - Prototypes
 - Releases

To understand the UML, you need to form a **conceptual model of the language**, and this requires learning three major elements:

1. Things
2. Relationships
3. Diagrams

Things in the UML

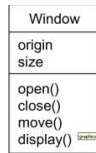
There are four kinds of things in the UML:

Structural things
Behavioral things
Grouping things
Annotational things

Structural things are the nouns of UML models. These are the mostly static parts of a model, representing elements that are either conceptual or physical. In all, there are seven kinds of structural things.

1. Classes
2. Interfaces
3. Collaborations
4. Use cases
5. Active classes
6. Components
7. Nodes

Class is a description of a set of objects that share the same attributes, operations, relationships, and semantics. A class implements one or more interfaces. Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations.



Interface

Interface is a collection of operations that specify a service of a class or component.

An interface therefore describes the externally visible behavior of that element.

An interface might represent the complete behavior of a class or component or only a part of that behavior.

An interface is rendered as a circle together with its name. An interface rarely stands alone. Rather, it is typically attached to the class or component that realizes the interface



Collaboration defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements. Therefore, collaborations have structural, as well as behavioral, dimensions. A given class might participate in several collaborations.

Graphically, a collaboration is rendered as an ellipse with dashed lines, usually including only its name

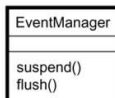


Usecase

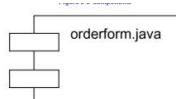
- Use case is a description of set of sequence of actions that a system performs that yields an observable result of value to a particular actor
- Use case is used to structure the behavioral things in a model.
- A use case is realized by a collaboration. Graphically, a use case is rendered as an ellipse with solid lines, usually including only its name



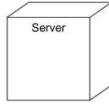
Active class is just like a class except that its objects represent elements whose behavior is concurrent with other elements. Graphically, an active class is rendered just like a class, but with heavy lines, usually including its name, attributes, and operations



Component is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces. Graphically, a component is rendered as a rectangle with tabs



Node is a physical element that exists at run time and represents a computational resource, generally having at least some memory and often processing capability. Graphically, a node is rendered as a cube, usually including only its name



Behavioral Things are the dynamic parts of UML models. These are the verbs of a model, representing behavior over time and space. In all, there are two primary kinds of behavioral things

Interaction
state machine

Interaction

Interaction is a behavior that comprises a set of messages exchanged among a set of objects within a particular context to accomplish a specific purpose

An interaction involves a number of other elements, including messages, action sequences and links

Graphically a message is rendered as a directed line, almost always including the name of its operation

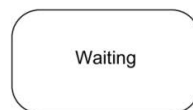


State Machine

State machine is a behavior that specifies the sequences of states an object or an interaction goes through during its lifetime in response to events, together with its responses to those events

State machine involves a number of other elements, including states, transitions, events and activities

Graphically, a state is rendered as a rounded rectangle, usually including its name and its substates

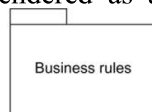


Grouping Things:-

1. are the organizational parts of UML models. These are the boxes into which a model can be decomposed
2. There is one primary kind of grouping thing, namely, packages.

Package:-

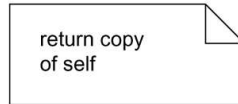
- A package is a general-purpose mechanism for organizing elements into groups. Structural things, behavioral things, and even other grouping things may be placed in a package
- Graphically, a package is rendered as a tabbed folder, usually including only its name and, sometimes, its contents



Annotational things are the explanatory parts of UML models. These are the comments you may apply to describe about any element in a model.

A **note** is simply a symbol for rendering constraints and comments attached to an element or a collection of elements.

Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment



Relationships in the UML: There are four kinds of relationships in the UML:

1. Dependency
2. Association
3. Generalization
4. Realization

Dependency:-

Dependency is a semantic relationship between two things in which a change to one thing may affect the semantics of the other thing

Graphically a dependency is rendered as a dashed line, possibly directed, and occasionally including a label



Association is a structural relationship that describes a set of links, a link being a connection among objects.

Graphically an association is rendered as a solid line, possibly directed, occasionally including a label, and often containing other adornments, such as multiplicity and role names



Aggregation is a special kind of association, representing a structural relationship between a whole and its parts. Graphically, a generalization relationship is rendered as a solid line with a hollow arrowhead pointing to the parent



Realization is a semantic relationship between classifiers, wherein one classifier specifies a contract that another classifier guarantees to carry out. Graphically a realization relationship is rendered as a cross between a generalization and a dependency relationship



Diagrams in the UML

- **Diagram** is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).
- In theory, a diagram may contain any combination of things and relationships.
- For this reason, the UML includes nine such diagrams:
 - Class diagram
 - Object diagram
 - Use case diagram
 - Sequence diagram
 - Collaboration diagram
 - Statechart diagram
 - Activity diagram
 - Component diagram
 - Deployment diagram

Class diagram

A class diagram shows a set of classes, interfaces, and collaborations and their relationships.

Class diagrams that include active classes address the static process view of a system.

Object diagram

- Object diagrams represent static snapshots of instances of the things found in class diagrams
- These diagrams address the static design view or static process view of a system
- An object diagram shows a set of objects and their relationships

Use case diagram

- A use case diagram shows a set of use cases and actors and their relationships
- Use case diagrams address the static use case view of a system.
- These diagrams are especially important in organizing and modeling the behaviors of a system.

Interaction Diagrams

Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams

Interaction diagrams address the dynamic view of a system

A sequence diagram is an interaction diagram that emphasizes the time-ordering of messages

A collaboration diagram is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages

Sequence diagrams and collaboration diagrams are isomorphic, meaning that you can take one and transform it into the other

Statechart diagram

- A statechart diagram shows a state machine, consisting of states, transitions, events, and activities
- Statechart diagrams address the dynamic view of a system
- They are especially important in modeling the behavior of an interface, class, or collaboration and emphasize the event-ordered behavior of an object

Activity diagram

An activity diagram is a special kind of a statechart diagram that shows the flow from activity to activity within a system

Activity diagrams address the dynamic view of a system

They are especially important in modeling the function of a system and emphasize the flow of control among objects

Component diagram

- A component diagram shows the organizations and dependencies among a set of components.
- Component diagrams address the static implementation view of a system
- They are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations

Deployment diagram

- A deployment diagram shows the configuration of run-time processing nodes and the components that live on them
- Deployment diagrams address the static deployment view of an architecture

Rules of the UML

The UML has semantic rules for

- | | |
|---------------|--|
| 1. Names | What you can call things, relationships, and diagrams |
| 2. Scope | The context that gives specific meaning to a name |
| 3. Visibility | How those names can be seen and used by others |
| 4. Integrity | How things properly and consistently relate to one another |
| 5. Execution | What it means to run or simulate a dynamic model |

Structural Diagrams

- The UML's four structural diagrams exist to visualize, specify, construct, and document the static aspects of a system.
- The UML's structural diagrams are roughly organized around the major groups of things you'll find when modeling a system.
 - Class diagram : Classes, interfaces, and collaborations
 - Object diagram : Objects
 - Component diagram : Components
 - Deployment diagram : Nodes

Class Diagram

- We use class diagrams to illustrate the static design view of a system.
- Class diagrams are the most common diagram found in modeling object-oriented systems.
- A class diagram shows a set of classes, interfaces, and collaborations and their relationships.
- Class diagrams that include active classes are used to address the static process view of a system.

Object Diagram

- Object diagrams address the static design view or static process view of a system just as do class diagrams, but from the perspective of real or prototypical cases.
- An object diagram shows a set of objects and their relationships.
- You use object diagrams to illustrate data structures, the static snapshots of instances of the things found in class diagrams.

Component Diagram

- We use component diagrams to illustrate the static implementation view of a system.
- A component diagram shows a set of components and their relationships.
- Component diagrams are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations.

Deployment Diagram

- We use deployment diagrams to illustrate the static deployment view of an architecture.
- A deployment diagram shows a set of nodes and their relationships.
- Deployment diagrams are related to component diagrams in that a node typically encloses one or more components.

Behavioral Diagrams

- The UML's five behavioral diagrams are used to visualize, specify, construct, and document the dynamic aspects of a system.
- The UML's behavioral diagrams are roughly organized around the major ways you can model the dynamics of a system.
 - Use case diagram : Organizes the behaviors of the system
 - Sequence diagram : Focused on the time ordering of messages
 - Collaboration diagram : Focused on the structural organization of objects that send and receive messages
 - Statechart diagram : Focused on the changing state of a system driven by events
 - Activity diagram : Focused on the flow of control from activity to activity

Use Case Diagram

- A use case diagram shows a set of use cases and actors and their relationships.
- We apply use case diagrams to illustrate the static use case view of a system.
- Use case diagrams are especially important in organizing and modeling the behaviors of a system.

Sequence Diagram

- We use sequence diagrams to illustrate the dynamic view of a system.
- A sequence diagram is an interaction diagram that emphasizes the time ordering of messages.
- A sequence diagram shows a set of objects and the messages sent and received by those objects.
- The objects are typically named or anonymous instances of classes, but may also represent instances of other things, such as collaborations, components, and nodes.

Collaboration Diagram

- We use collaboration diagrams to illustrate the dynamic view of a system.
- A collaboration diagram is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages.
- A collaboration diagram shows a set of objects, links among those objects, and messages sent and received by those objects.
- The objects are typically named or anonymous instances of classes, but may also represent instances of other things, such as collaborations, components, and nodes.

* Sequence and collaboration diagrams are isomorphic, meaning that you can convert from one to the other without loss of information.

Statechart Diagram

We use statechart diagrams to illustrate the dynamic view of a system.

They are especially important in modeling the behavior of an interface, class, or collaboration.

A statechart diagram shows a state machine, consisting of states, transitions, events, and activities.

Statechart diagrams emphasize the event-ordered behavior of an object, which is especially useful in modeling reactive systems.

Activity Diagram

We use activity diagrams to illustrate the dynamic view of a system.

Activity diagrams are especially important in modeling the function of a system.

Activity diagrams emphasize the flow of control among objects.

An activity diagram shows the flow from activity to activity within a system.

An activity shows a set of activities, the sequential or branching flow from activity to activity, and objects that act and are acted upon.

Class Diagrams

- A class diagram shows a set of classes, interfaces, and collaborations and their relationships.
- Graphically, a class diagram is a collection of vertices and arcs.

Common Properties

Contents

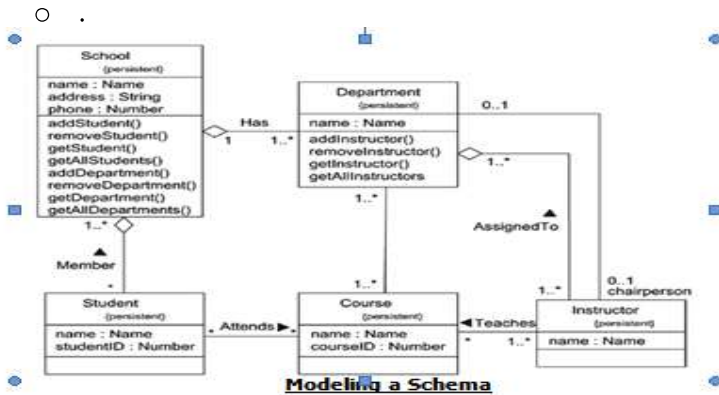
- Class diagrams commonly contain the following things:
 - Classes
 - Interfaces
 - Collaborations
 - Dependency, generalization, and association relationships
- Like all other diagrams, class diagrams may contain notes and constraints
- Class diagrams may also contain packages or subsystems

Note: Component diagrams and deployment diagrams are similar to class diagrams, except that

instead of containing classes, they contain components and nodes

Common Uses

- You use class diagrams to model the static design view of a system. This view primarily supports the functional requirements of a system



Object Diagram

- An object diagram is a diagram that shows a set of objects and their relationships at a point in time.
- Graphically, an object diagram is a collection of vertices and arcs
- An object diagram is a special kind of diagram and shares the same common properties as all other diagrams—that is, a name and graphical contents that are a projection into a model

Contents

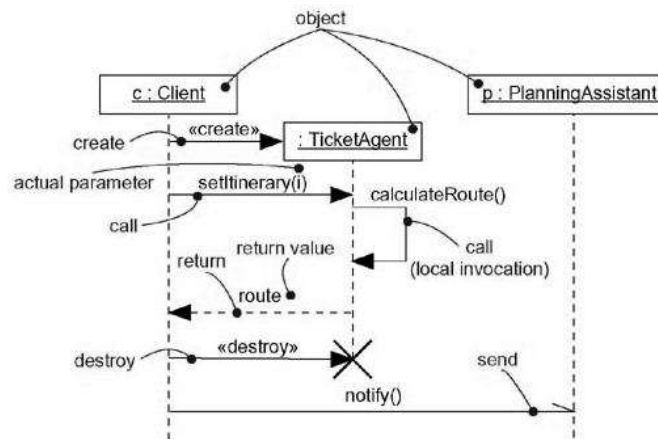
- Object diagrams commonly contain
 - Objects
 - Links
- Like all other diagrams, object diagrams may contain notes and constraints.
- Object diagrams may also contain packages or subsystems
-

Interactions

- An interaction is a behavior that comprises a set of messages exchanged among a set of objects within a context to accomplish a purpose.
- A message is a specification of a communication between objects that conveys information with the expectation that activity will ensue.

Context

- We can use interactions to visualize, specify, construct, and document the semantics of a class
- We may find an interaction wherever objects are linked to one another.
- We'll find interactions in the collaboration of objects that exist in the context of your system or subsystem.
- We will also find interactions in the context of an operation.
- We might create interactions that show how the attributes of that class collaborate with one another
- Finally, you'll find interactions in the context of a class.

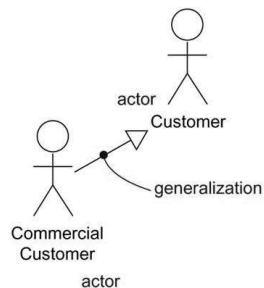


Use Cases

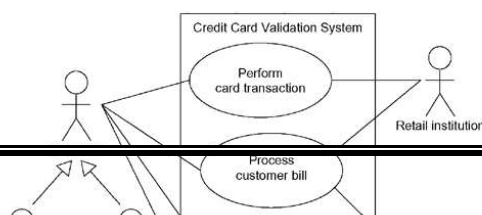
- A use case is a description of a set of sequences of actions, including variants, that a system performs to yield an observable result of value to an actor.
- Graphically, a use case is rendered as an ellipse.

Names

- Every use case must have a name that distinguishes it from other use cases. A name is a textual string.
- That name alone is known as a **simple name**; a **path name** is the use case name prefixed by the name of the package in which that use case lives.
- A use case is typically drawn showing only its name



Actors

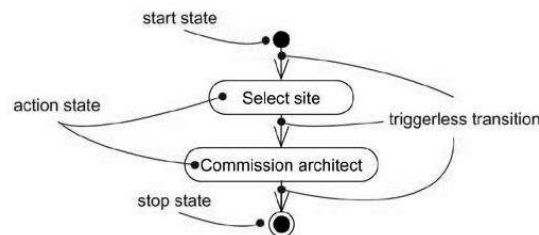


Activity Diagrams

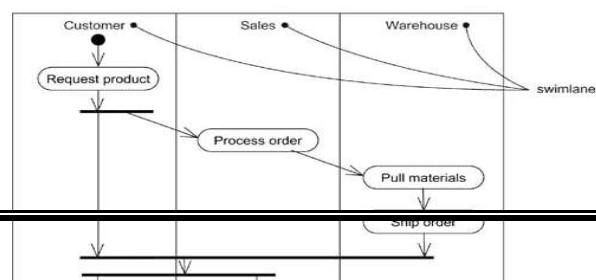
- An activity diagram shows the flow from activity to activity. An is an ongoing nonatomic execution within a state machine.
- Activities ultimately result in some action, which is made up of executable atomic computations that result in a change in state of the system or the return of a value.
- Actions encompass calling another operation, sending a signal, creating or destroying an object, or some pure computation, such as evaluating an expression.
- Graphically, an activity diagram is a collection of vertices and arcs.

Contents

- Activity diagrams commonly contain
 - Activity states and action states
 - Transitions
 - Objects
- Like all other diagrams, activity diagrams may contain notes and constraints.



Triggerless Transitions



State Machines

- **A state machine** is a behavior that specifies the sequences of states an object goes through during its lifetime in response to events, together with its responses to those events
- **A state** is a condition or situation during the life of an object during which it satisfies some condition, performs some activity, or waits for some event.
- **An event** is the specification of a significant occurrence that has a location in time and space.
- In the context of state machines, an event is an occurrence of a stimulus that can trigger a state transition
- **A transition** is a relationship between two states indicating that an object in the first state will perform certain actions and enter the second state when a specified event occurs and specified conditions are satisfied.
- **An activity** is ongoing nonatomic execution within a state machine
- **An action** is an executable atomic computation that results in a change in state of the model or the return of a value

Context

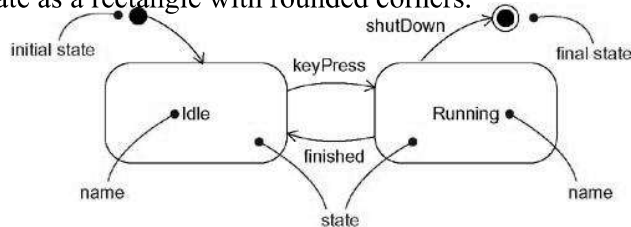
- Every object has a lifetime. On creation, an object is born; on destruction, an object ceases to exist.
- In between, an object may act on other objects (by sending them messages), as well as be acted on (by being the target of a message).
- In other kinds of systems, you'll encounter objects that must respond to signals, which are asynchronous stimuli communicated between instances.
- The behavior of objects that must respond to asynchronous stimulus or whose current behavior depends on their past is best specified by using a state machine
- This encompasses instances of classes that can receive signals, including many active objects. In fact, an object that receives a signal but has no state machine will simply ignore that signal.
- We'll also use state machines to model the behavior of entire systems, especially reactive systems, which must respond to signals from actors outside the system.

States

- A state is a condition or situation during the life of an object during which it satisfies some condition, performs some activity, or waits for some event.
- An object remains in a state for a finite amount of time.
- When an object's state machine is in a given state, the object is said to be in that state. A state has several parts.

1. Name	A textual string that distinguishes the state from other states; a state may be anonymous, meaning that it has no name
2. Entry/exit actions	Actions executed on entering and exiting the state, respectively
3. Internal transitions	Transitions that are handled without causing a change in state
4. Substates	The nested structure of a state, involving disjoint (sequentially active) or concurrent (concurrently active) substates
5. Deferred events	A list of events that are not handled in that state but, rather, are postponed and queued for handling by the object in another state

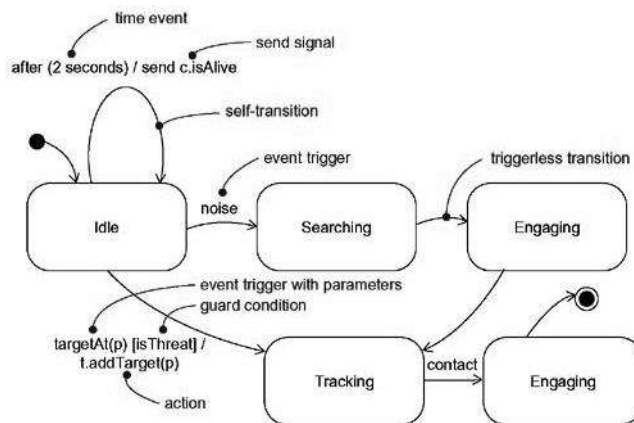
We represent a state as a rectangle with rounded corners.



States

Initial and Final States

- There are two special states that may be defined for an object's state machine.
- **initial state**, which indicates the default starting place for the state machine or substate.
- An initial state is represented as a filled black circle
- **final state**, which indicates that the execution of the state machine or the enclosing state has been completed.
- A final state is represented as a filled black circle surrounded by an unfilled circle.
- .



UNIT III